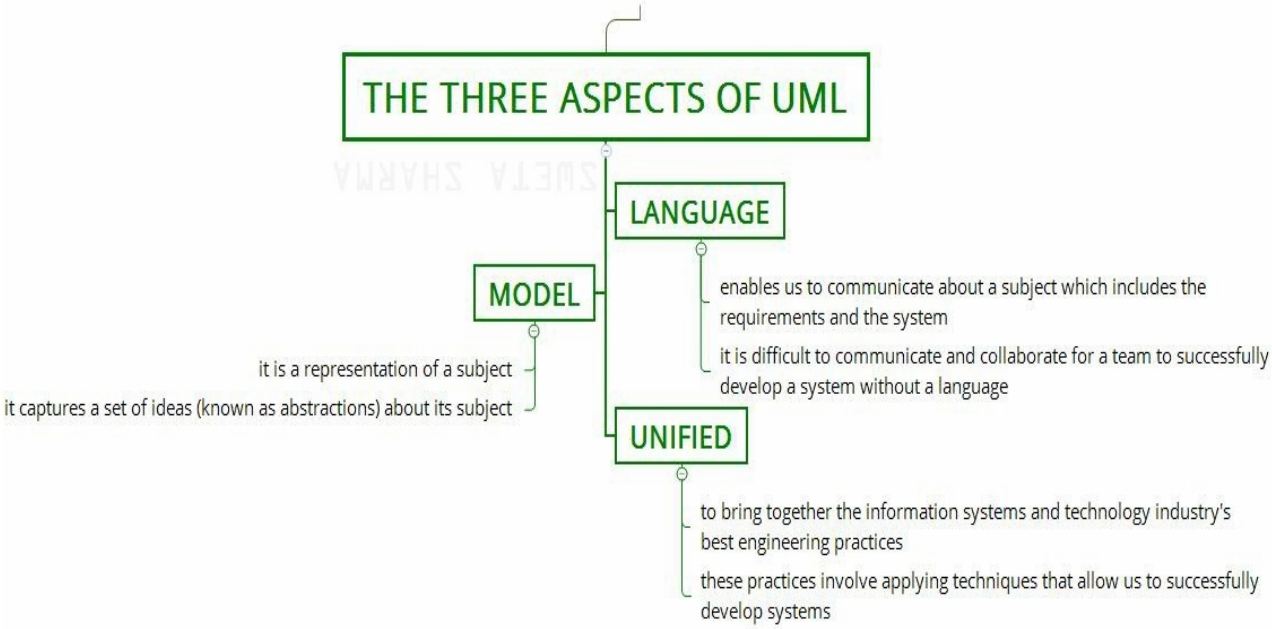
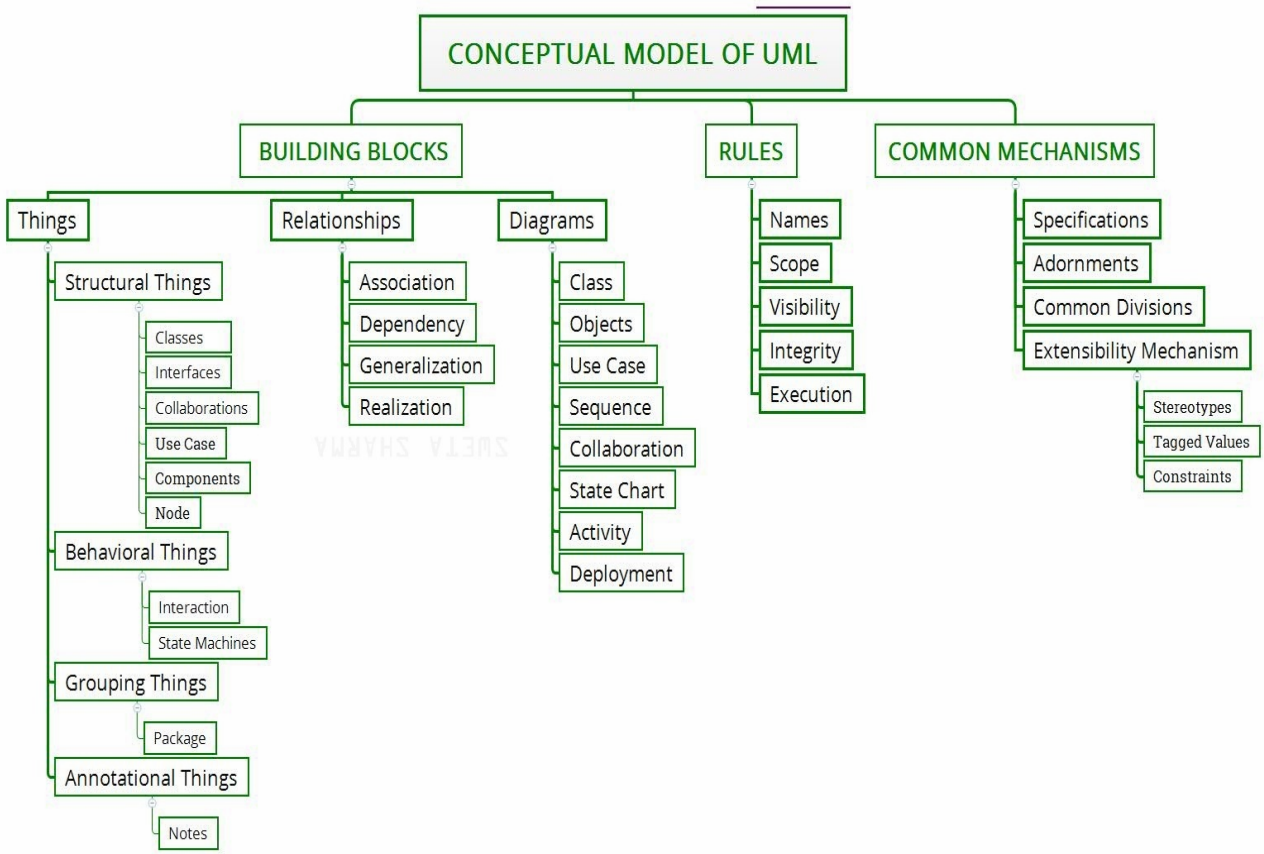
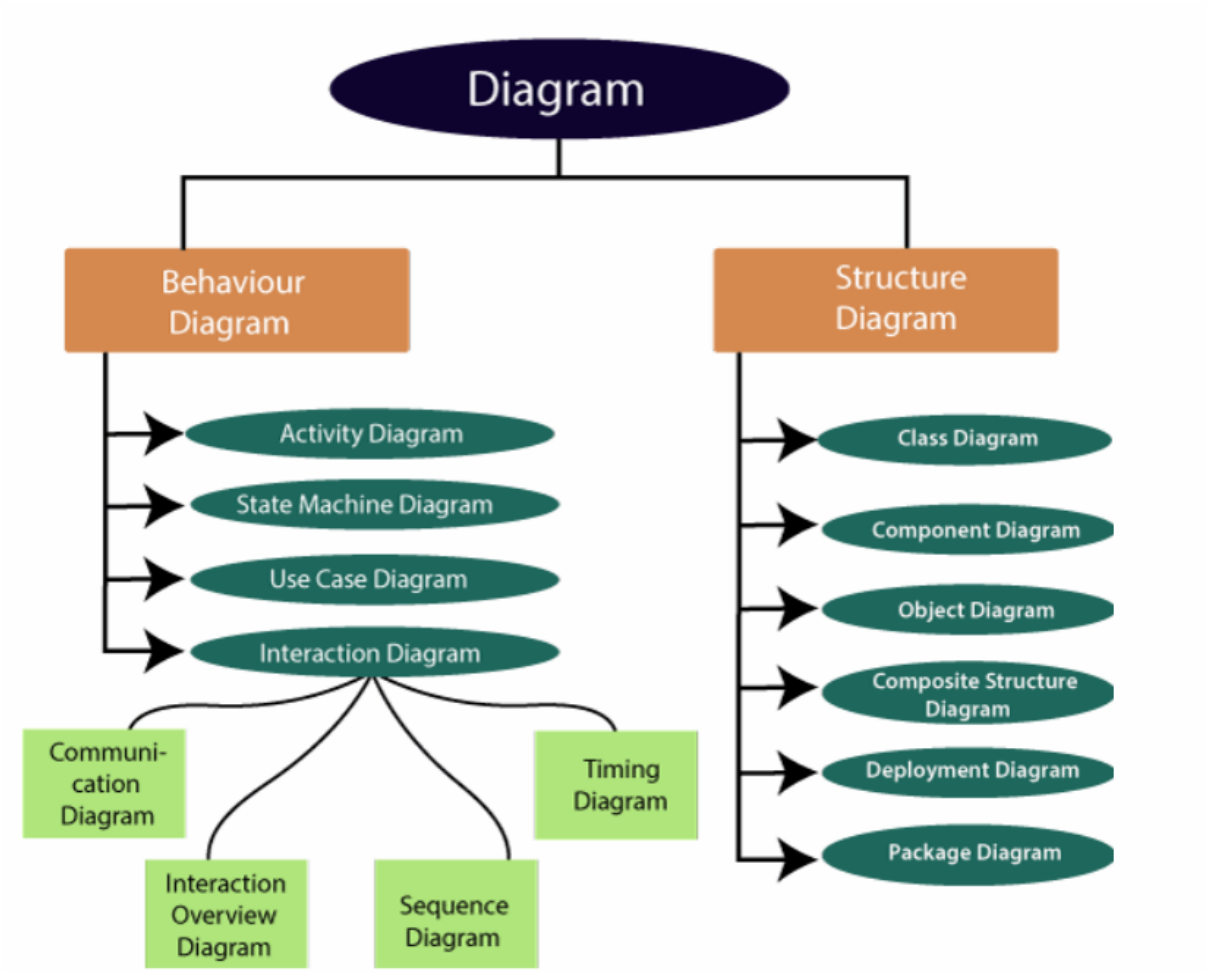
CASE TOOLS LAB MANUAL







Unified Modeling Language (UML)

Unified Modeling Language -- A system that allows a software designer to graphically layout and model a software application. It gives designers a way to literally draw a map of how a piece of software will be constructed and function.

A Brief History

Flow Charts

Prior to the mid 1970s, software design, if there was any, may have been aided by the use of Flow Charts. These were diagrams of how the various parts of a program interacted. It was

primarily Procedural Oriented as opposed to Object Oriented. A Flow Chart displayed graphically the "flow of control" through a program.

Object Oriented Modeling Languages

From the mid 1970s to the late 1980s a number of new software "modeling languages" appeared. The three developers who stand out for their work during this period are: (1) **Grady Booch** of Rational Software Corporation who developed the **Booch Method**, (2) **Ivan Jacobson** of Objectory, Inc. who developed **Object Oriented Software Engineering**, and (3) **James Rumbauch** of General Electric who developed the **Object Modeling Technique**.

Unified Modeling Language

Booch, Jacobson, and Rumbaugh were independently working toward the same objectives. In the mid 1990s they began collaborating. The result was the **Unified Modeling Language** of which the first version (then called just the Unified Method) was released in October of 1995.

Why Use UML?

To be successful a software company must produce quality software, that meets

requirements and is released on time and within budget.

This was easy to do when computers were very simple and slow devices and programs

did little more than play solitaire, convert between English and Metric measures, and balance

your checkbook.

Today when a program may be called upon to "simulate a battlefield scenario with full and

accurate engineering and physics based rendering" this can be very difficult if not impossible without some means of organizing the design phase.

Modeling a software system using UML is a lot like drawing up the blueprints to build a home. Would you want the builders to just go to your site and just start "hacking together" your home?

Getting started with UML

UML is not difficult to learn. It consists of three parts.

1. The basic building blocks (the words of the language)
2. Rules controlling how the blocks are put together (the syntax)
3. Common mechanisms that apply throughout the language (use conventions or semantics)

This brief introduction to UML just covers the basic building block of UML.

Basic Building Blocks of UML

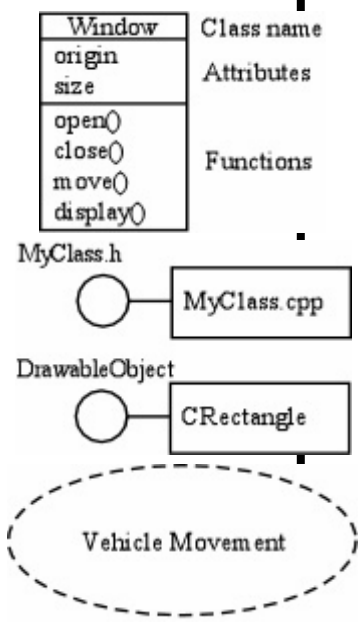
Three Types

1. **Things**
   Four Types
   - Structural things -- The nouns of UML models. These represent elements that are conceptual or physical. There are seven kinds of structural things: Class, Interface, Collaboration, Use Case, Active Class, Component, and Node. (See the examples below.)
   - Behavioral things -- Dynamic parts of UML models. The verbs which represent behavior over time and space. There are two kinds of behavioral things: Interaction, and State Machine. (See the examples below.)
   - Grouping things -- Organizational parts of UML. These are boxes into which models can be decomposed. There is only one kind of grouping thing, the Package. (See the examples below.)
   - Annotational things -- Explanatory parts of UML. Used to describe, illuminate, and remark any element of a model. There is only one kind of annotational thing, the Note. (See the examples below.)
2. **Relationships**
   Four Types
   - Dependency -- A semantic relationship in which a change on one thing (the independent thing) may cause changes in the other thing (the dependent thing). (See the examples below.)
   - Association -- A structural relationship describing links between objects. May also include labels to indicate number and role of the links. In the example shown below there may be any number of employees (*) each of which has 0 or 1 employer. (See the examples below.)
   - Generalization -- A specialization/generalization relationship. Simply put this describes the relationship of a parent class (generalization) to its subclasses (specializations). (See the examples below.)
   - Realization -- Defines a relationship in which one class specifies something that another class will perform. Example: The relationship between an interface and the class that realizes or executes that interface. (See the examples below.)
3. **Diagrams**
   Nine Types
   - Class Diagram -- A set of classes, interfaces, and collaborations and their relationships. Most often found in modeling Object Oriented systems. (See the examples below.)

- Object Diagram -- A set of objects and their relationships. Represents static instances of things found in class diagrams. (See the examples below.)
- Use Case Diagram -- A set of Use Cases and actors. (See the examples below.)
- Sequence Diagram -- An interactive diagram (set of objects, relationships, and messages that may be exchanged) emphasizing the time-ordering of messages. (See the examples below.)
- Collaboration Diagram -- An interaction diagram emphasizes the structural organization of the objects that send and receive messages. (See the examples below.)
- Statechart Diagram -- Shows a state machine with states, transitions, events, and activities. (See the examples below.)
- Activity Diagram -- Special type of statechart diagram that shows the flow from activity to activity within a system. (See the examples below.)
- Component Diagram -- Shows the organizations and dependencies among a set of components. (See the examples below.)
- Deployment Diagram -- Shows the configuration of run-time processing nodes and the components that are part of them. (See the examples below.)

Examples of the Basic Building Blocks of UML

Seven Types of Structural Things



1. Class - An object with defined attributes and operations. A class in UML is very much like a class in C++.

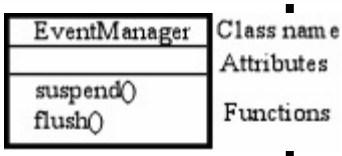
2. Interface - A collection of functions that specify a service of a class or component, i.e. externally visible behavior of that class.

3. Collaboration - A larger pattern of behaviors and actions. Example: All classes and behaviors that create the modeling of a moving tank in a simulation.

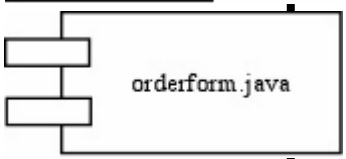
4. Use Case - A sequence of actions that a system performs that yields an observable result. Used to structure behavior in a model. Is realized by a

collaboration.

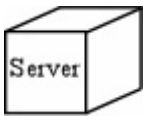
5. Active Class - Like a class but its represents behavior that runs concurrent with other behaviors, i.e. threading.



6. Component - A physical and replaceable part of a system that implements a number of interfaces. Example: a set of classes, interfaces, and collaborations.

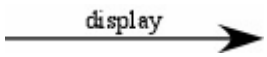


7. Node - A physical element existing at run time and represents a resource.

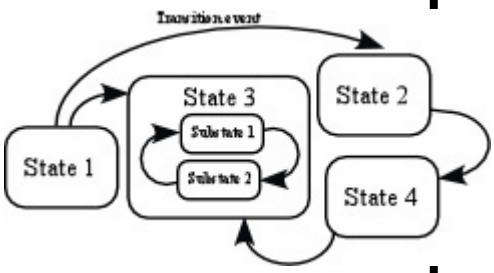


Two Types of Behavioral Things

1. Interaction - A behavior made up of a set of messages exchanged among a set of objects in a particular context to accomplish a specific purpose.

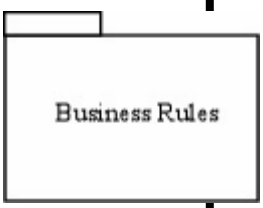


2. State Machine - A behavior that specifies the sequences of states an object or interaction goes through during its' lifetime in response to events.



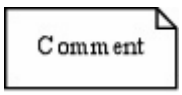
One Type of Grouping Thing

1. Package - A general purpose mechanism for organizing elements into groups.



One Type of Annotation Thing

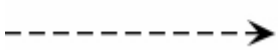
1. Note - A symbol to display comments.

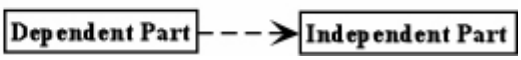


Four Types of Relationships

1. Dependency - A semantic relationship in which a change on one thing (the independent thing) may cause changes in the other thing (the dependent thing).

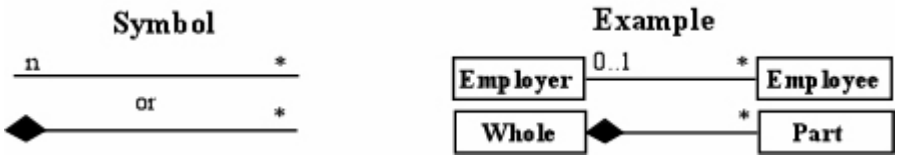
Symbol **Example**



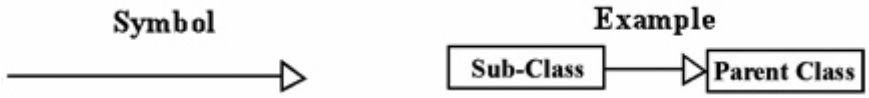


2. Association - A structural relationship describing links between objects. May also include labels to

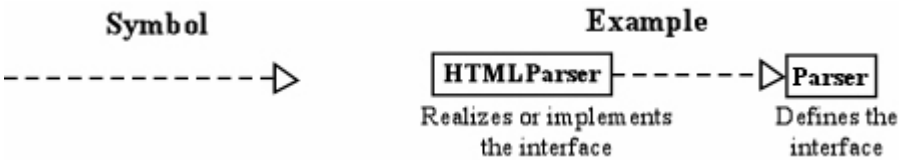
indicate number and role of the links. In the example there may be any number of employees (*) each of which has 0 or 1 employer. The double arrowhead is used to indicate a "has-a" relationship, meaning there is 1 employer who may have many (*) employees.

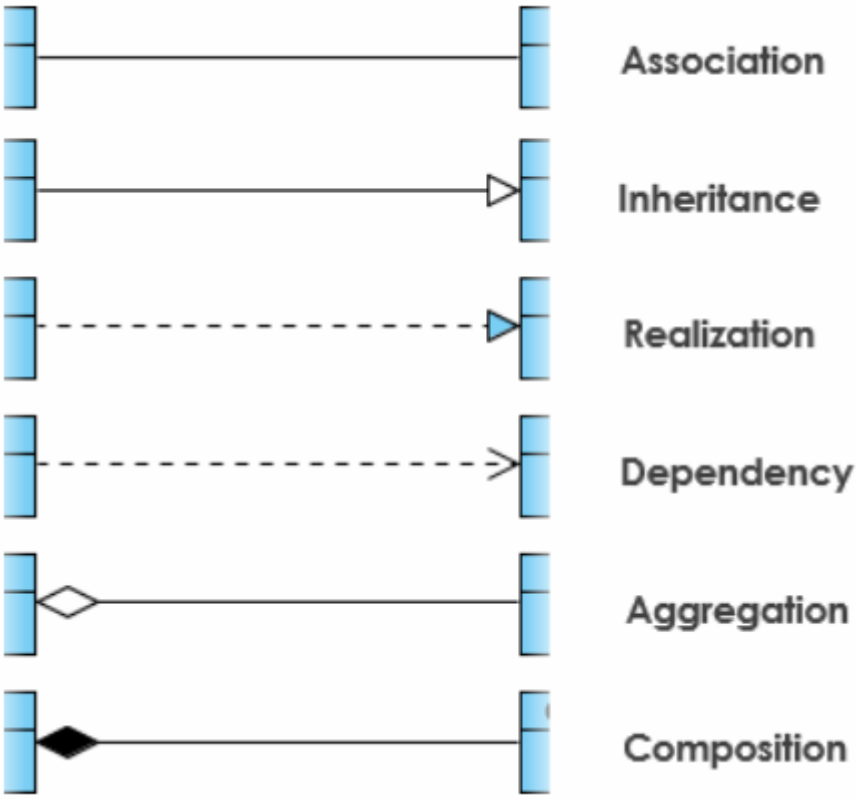


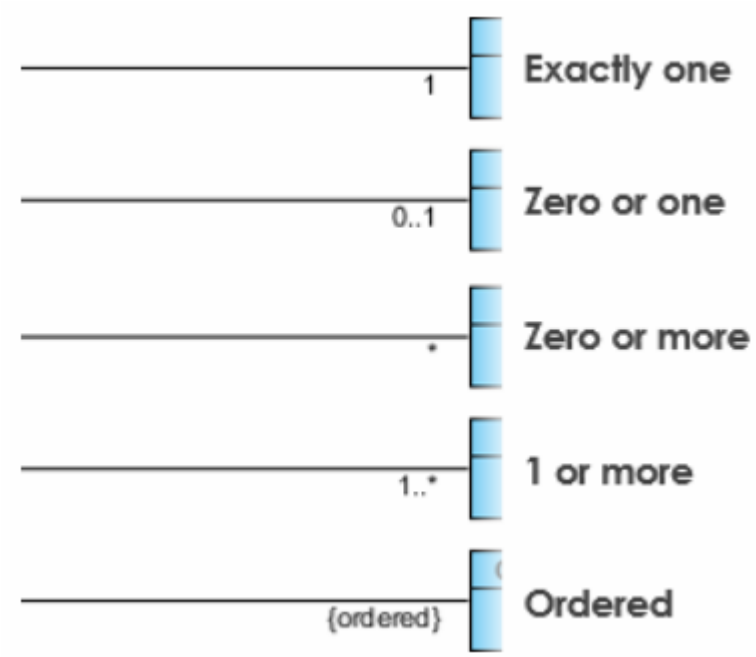
3. Generalization - A specialization/generalization relationship. Simply put this describes the relationship of a parent class (generalization) to its subclasses (specializations).



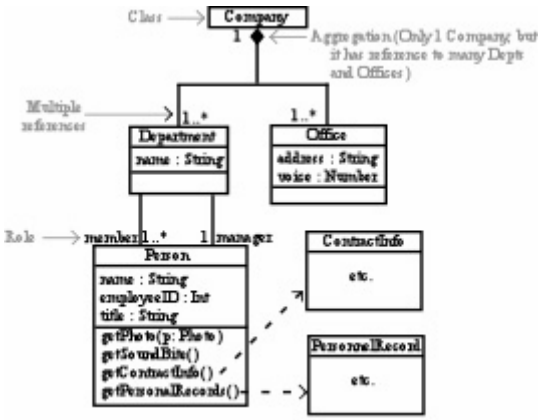
4. Realization - Defines a relationship in which one class specifies something that another class will perform. Example: The relationship between an interface and the class that realizes or executes that interface.



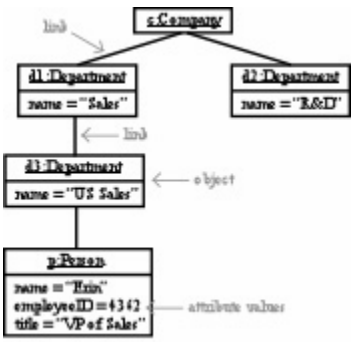




Nine Types of Diagrams



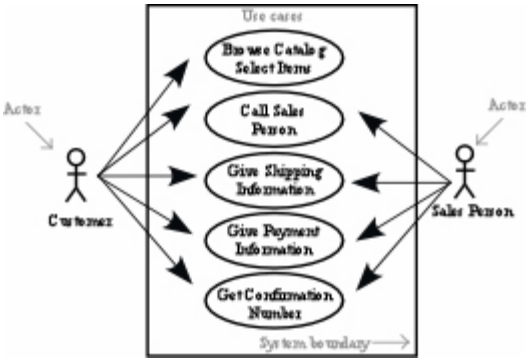
1. Class Diagram - A set of classes, interfaces, and collaborations and their relationships. Most often found in modelling OO systems.

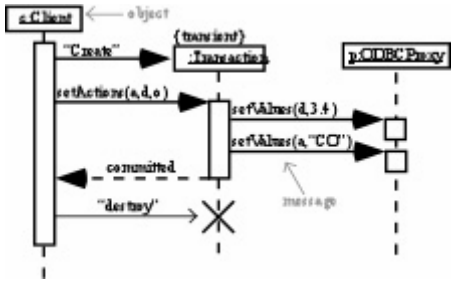


2. Object Diagram - A set of objects and their relationships. Represents static instances of things found in class diagrams.

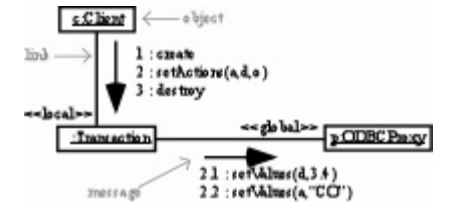
3. Use Case Diagram - A set of Use Cases and actors.

4. Sequence Diagram - An interactive diagram (set of objects, relationships, and messages that may be exchanged) emphasizing the time-ordering of messages.

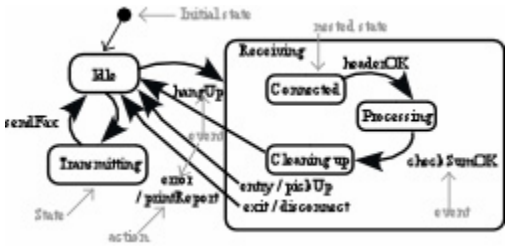




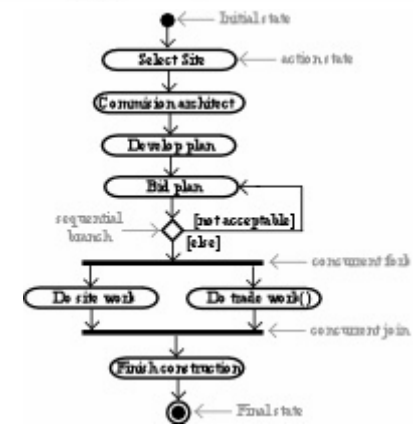
5. Collaboration Diagram - An interaction diagram emphasizes the structural organization of the objects that send and receive messages.



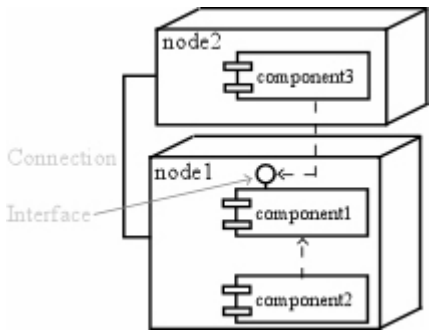
6. Statechart Diagram - Shows a state machine with states, transitions, events, and activities.



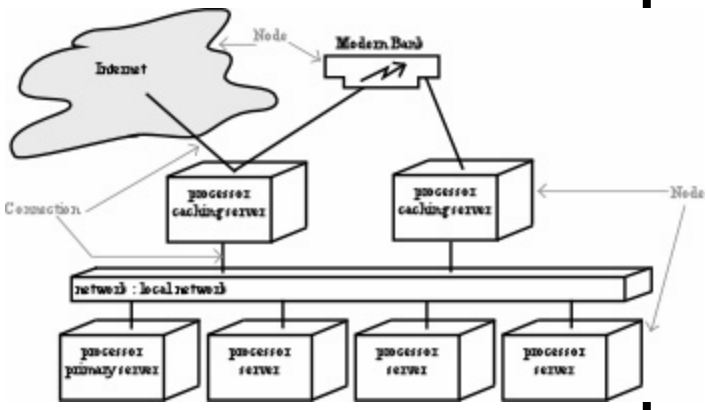
7. Activity Diagram - Special type of statechart diagram that shows the flow from activity to activity within a system.



8. Component Diagram - Shows the organizations and dependencies among a set of components.



9. Deployment Diagram - Shows the configuration of run-time processing nodes and the components that are part of them.



List of Experiments (CASE Tools):

I. ATM Case Study

1. Use Case diagram for ATM system, with use case specification.

Theory:

Implementation:

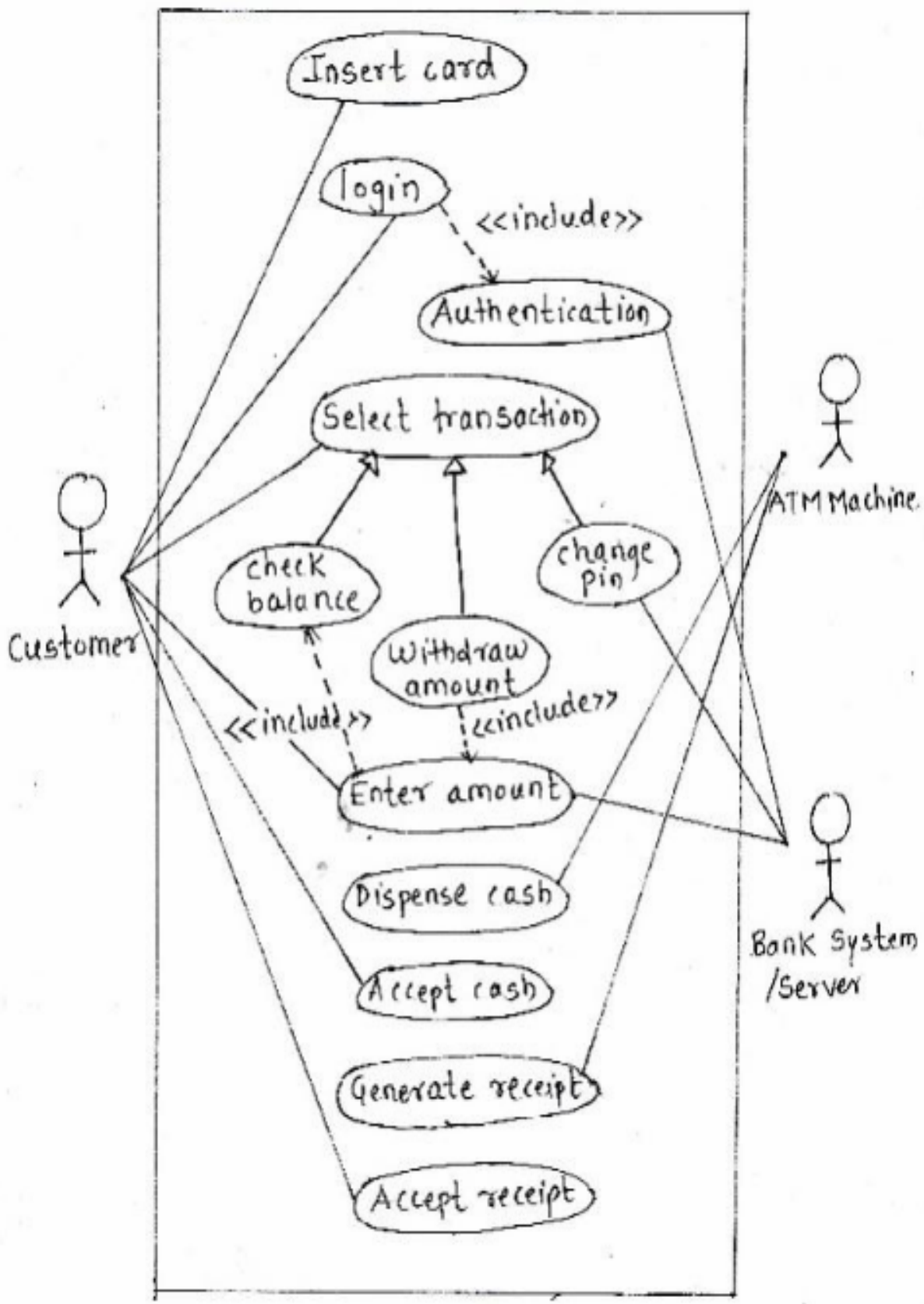
1. **Diagram/Architecture:**

2. **Steps**

Use Case Specifications:

1. **Use Case:** Withdraw Money
   - **Description:** Allows a user to withdraw a specific amount of money from their account.
   - **Actors:** User
   - **Preconditions:** User must have a valid account and sufficient balance.
   - **Postconditions:** User's account balance is updated, and the ATM dispenses the requested amount of cash.
   - **Basic Flow:**

        1. User inserts card into the ATM.
        2. System prompts for a PIN.
        3. User enters the PIN.
        4. System validates the PIN and presents the withdrawal options.
        5. User selects an amount to withdraw.
        6. System verifies the balance and dispenses cash.
        7. User takes the cash and card.
    - **Alternate Flows:**
        1. *Invalid PIN:* If the entered PIN is invalid, the system informs the user and prompts them to re-enter the PIN.
2. **Use Case:** Check Balance
    - **Description:** Allows a user to check the current balance of their account.
    - **Actors:** User
    - **Preconditions:** User must have a valid account.
    - **Postconditions:** None
    - **Basic Flow:**
        1. User inserts card into the ATM.
        2. System prompts for a PIN.
        3. User enters the PIN.
        4. System validates the PIN and displays the account balance.
        5. User takes the card.
    - **Alternate Flows:**
        1. *Invalid PIN:* If the entered PIN is invalid, the system informs the user and prompts them to re-enter the PIN.


Insert card
login
<<include>>
Authentication
Select transaction
check balance
Withdraw amount
change pin
<<include>>
<<include>>
Enter amount
Dispense cash
Accept cash
Generate receipt
Accept receipt
Customer
ATM Machine
Bank System /Server

3. **Identify the classes for the ATM system and draw the Class Diagram.**

Theory:

Basic components of a class diagram

The standard class diagram is composed of three sections:

- Upper section: Contains the name of the class. This section is always required, whether you are talking about the classifier or an object.
- Middle section: Contains the attributes of the class. Use this section to describe the qualities of the class. This is only required when describing a specific instance of a class.
- Bottom section: Includes class operations (methods). Displayed in list format, each operation takes up its own line. The operations describe how a class interacts with data.

Member access modifiers

All classes have different access levels depending on the access modifier (visibility). Here are the access levels with their corresponding symbols:

- Public (+)
- Private (-)
- Protected (#)
- Package (~)
- Derived (/)
- Static (underlined)

Member scopes

There are two scopes for members: classifiers and instances.

Classifiers are static members while instances are the specific instances of the class. If you are familiar with basic OO theory, this isn't anything groundbreaking.

Additional class diagram components

Depending on the context, classes in a class diagram can represent the main objects, interactions in the application, or classes to be programmed. To answer the question "What is a class diagram in UML?" you should first understand its basic makeup.

- Classes: A template for creating objects and implementing behavior in a system. In UML, a class represents an object or a set of objects that share a common structure and behavior. They're represented by a rectangle that includes rows of the class name, its attributes, and its operations. When you draw a class in a class diagram, you're only required to fill out the top row—the others are optional if you'd like to provide more detail.
    - Name: The first row in a class shape.
    - Attributes: The second row in a class shape. Each attribute of the class is displayed on a separate line.
    - Methods: The third row in a class shape. Also known as operations, methods are displayed in list format with each operation on its own line.
- Signals: Symbols that represent one-way, asynchronous communications between active objects.
- Data types: Classifiers that define data values. Data types can model both primitive types and enumerations.
- Packages: Shapes designed to organize related classifiers in a diagram. They are symbolized with a large tabbed rectangle shape.
- Interfaces: A collection of operation signatures and/or attribute definitions that define a cohesive set of behaviors. Interfaces are similar to classes, except that a class can have an

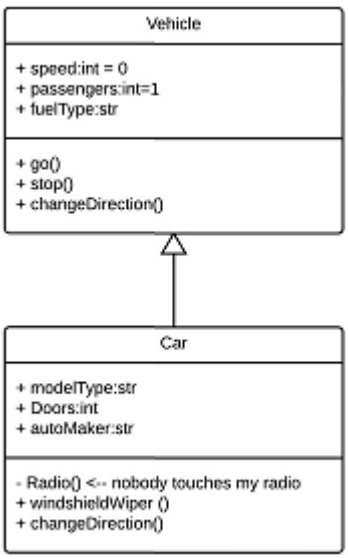
instance of its type, and an interface must have at least one class to implement it.

- Enumerations: Representations of user-defined data types. An enumeration includes groups of identifiers that represent values of the enumeration.
- Objects: Instances of a class or classes. Objects can be added to a class diagram to represent either concrete or prototypical instances.
- Artifacts: Model elements that represent the concrete entities in a software system, such as documents, databases, executable files, software components, etc.

Interactions

The term "interactions" refers to the various relationships and links that can exist in class and object diagrams. Some of the most common interactions include:

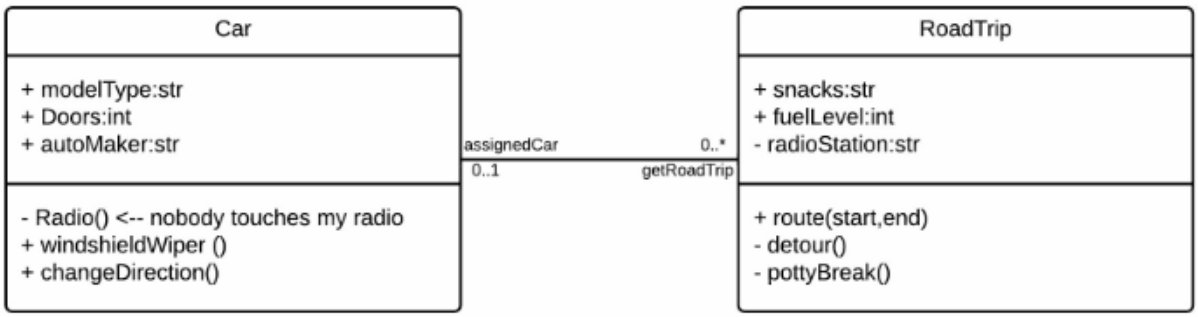
- Inheritance: The process of a child or sub-class taking on the functionality of a parent or superclass, also known as generalization. It's symbolized with a straight connected line with a closed arrowhead pointing towards the superclass.



In this example, the object "Car" would inherit all of the attributes (speed, numbers of passengers, fuel) and methods (go(), stop(), changeDirection()) of the parent class ("Vehicle") in addition

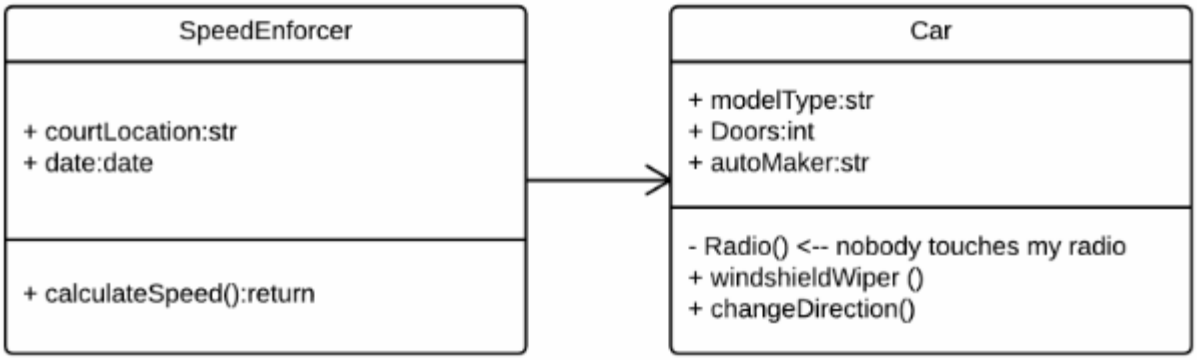
to the specific attributes (model type, number of doors, auto maker) and methods of its own class (Radio(), windshieldWiper(), ac/heat()). Inheritance is shown in a class diagram by using a solid line with a closed, hollow arrow.

- Bidirectional association: The default relationship between two classes. Both classes are aware of each other and their relationship with the other. This association is represented by a straight line between two classes.



In the example above, the Car class and RoadTrip class are interrelated. At one end of the line, the Car takes on the association of "assignedCar" with the multiplicity value of 0..1, so when the instance of RoadTrip exists, it can either have one instance of Car associated with it or no Cars associated with it. In this case, a separate Caravan class with a multiplicity value of 0..* is needed to demonstrate that a RoadTrip could have multiple instances of Cars associated with it. Since one Car instance could have multiple "getRoadTrip" associations—in other words, one car could go on multiple road trips—the multiplicity value is set to 0..*

- Unidirectional association: A slightly less common relationship between two classes. One class is aware of the other and interacts with it. Unidirectional association is modeled with a straight connecting line that points an open arrowhead from the knowing class to the known class.



As an example, on your road trip through Arizona, you might run across a speed trap where a speed cam records your driving activity, but you won't know about it until you get a notification in the mail. It isn't drawn in the image, but in this case, the multiplicity value would be 0..* depending on how many times you drive by the speed cam.

Implementation:

1. **Diagram/Architecture:**

2. **Steps**

Classes:

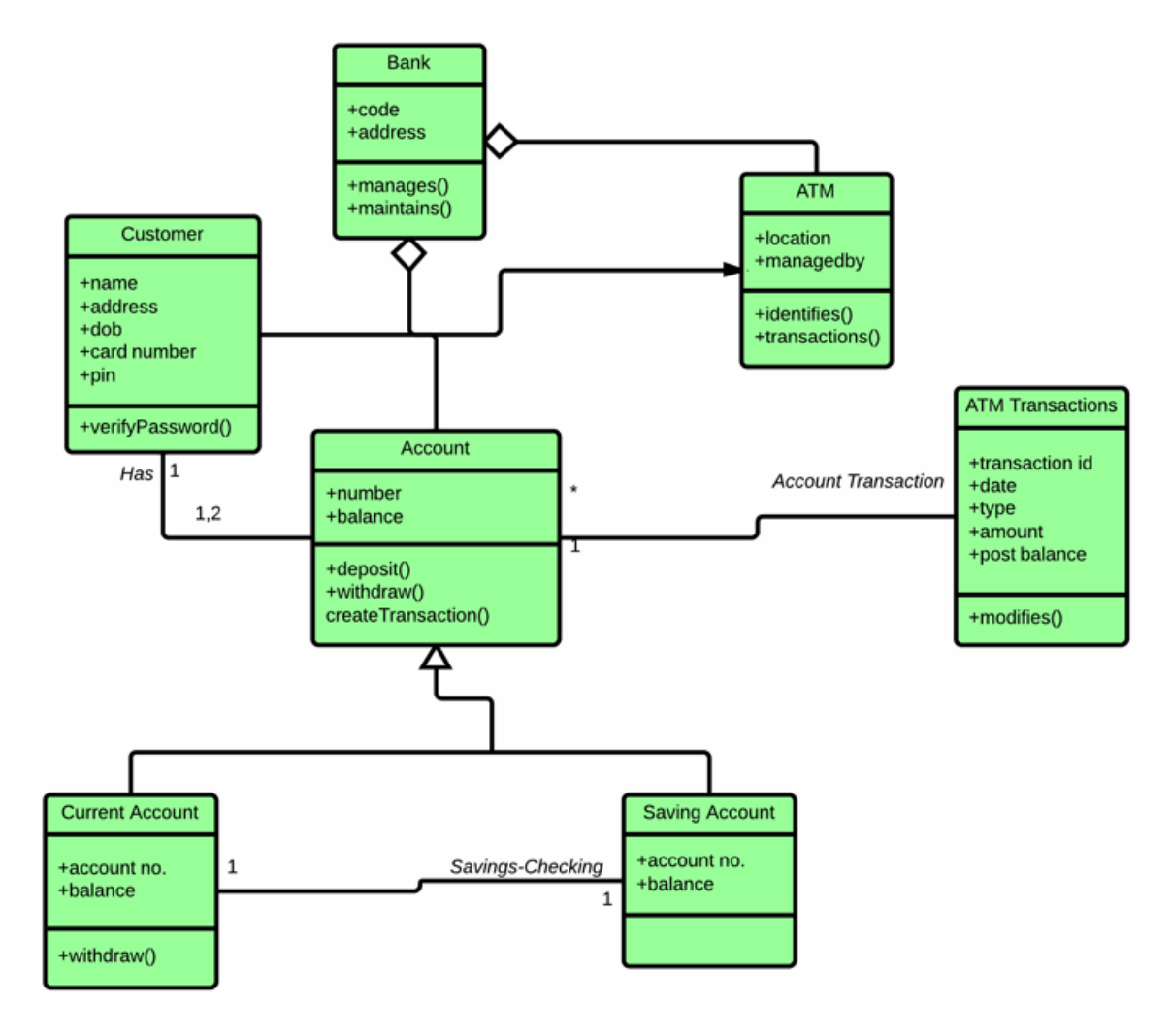
1. **ATM:**
   - **Properties: location, ID, cashDispenser, cardReader, ...**
2. **User:**
   - **Properties: userID, name, PIN, account(s), ...**
3. **Bank:**
   - **Properties: bankName, accountsDatabase, ...**
4. **Account:**
   - **Properties: accountNumber, balance, accountType, ...**
5. **CashDispenser:**
   - **Properties: cashOnHand**

- **Methods: dispenseCash(amount)**

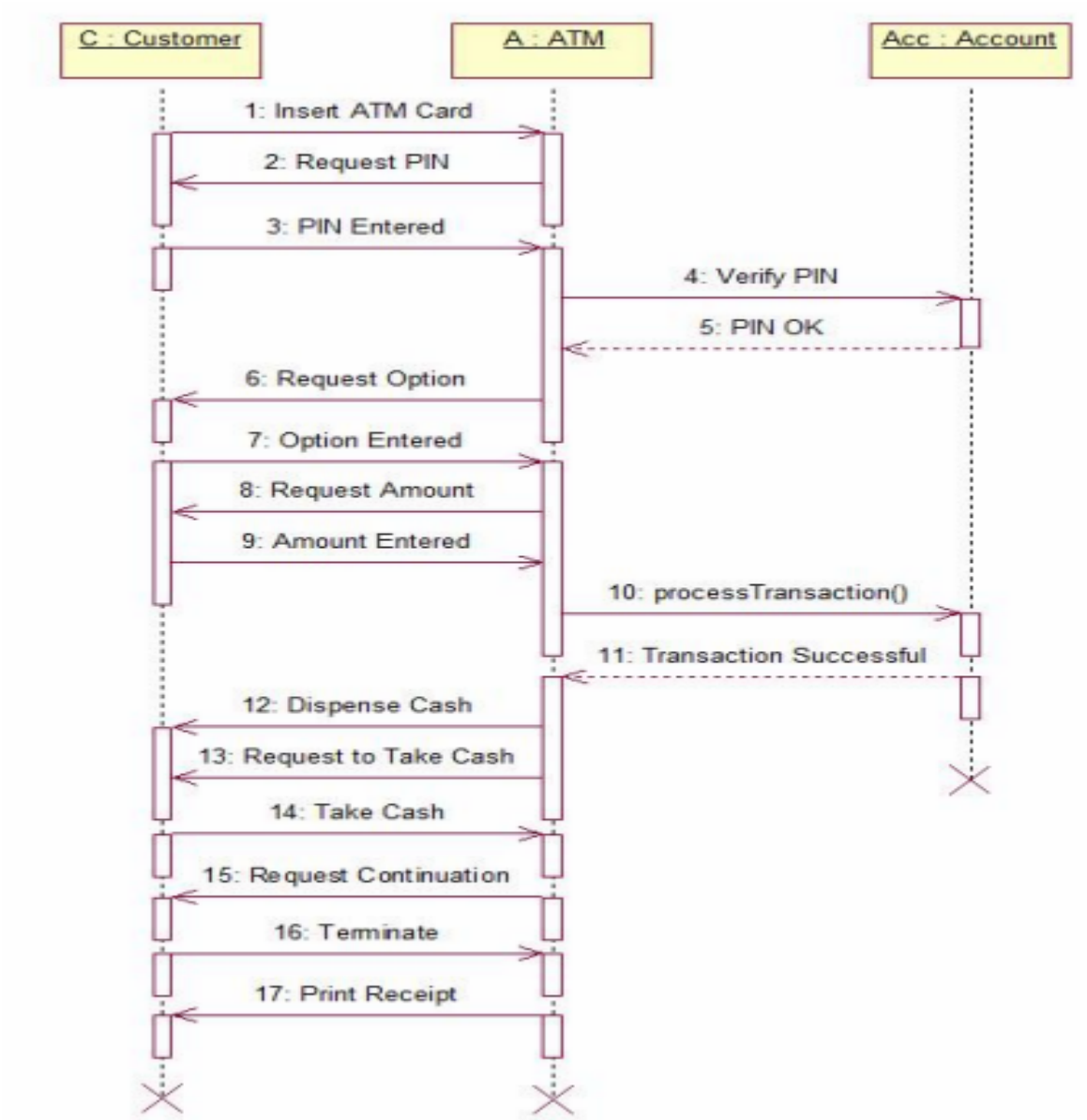
6. **CardReader:**
    - **Methods: readCard()**
7. **Transaction:**
    - **Properties: transactionID, timestamp, type, amount, ...**
8. **Screen:**
    - **Methods: displayMessage(message)**

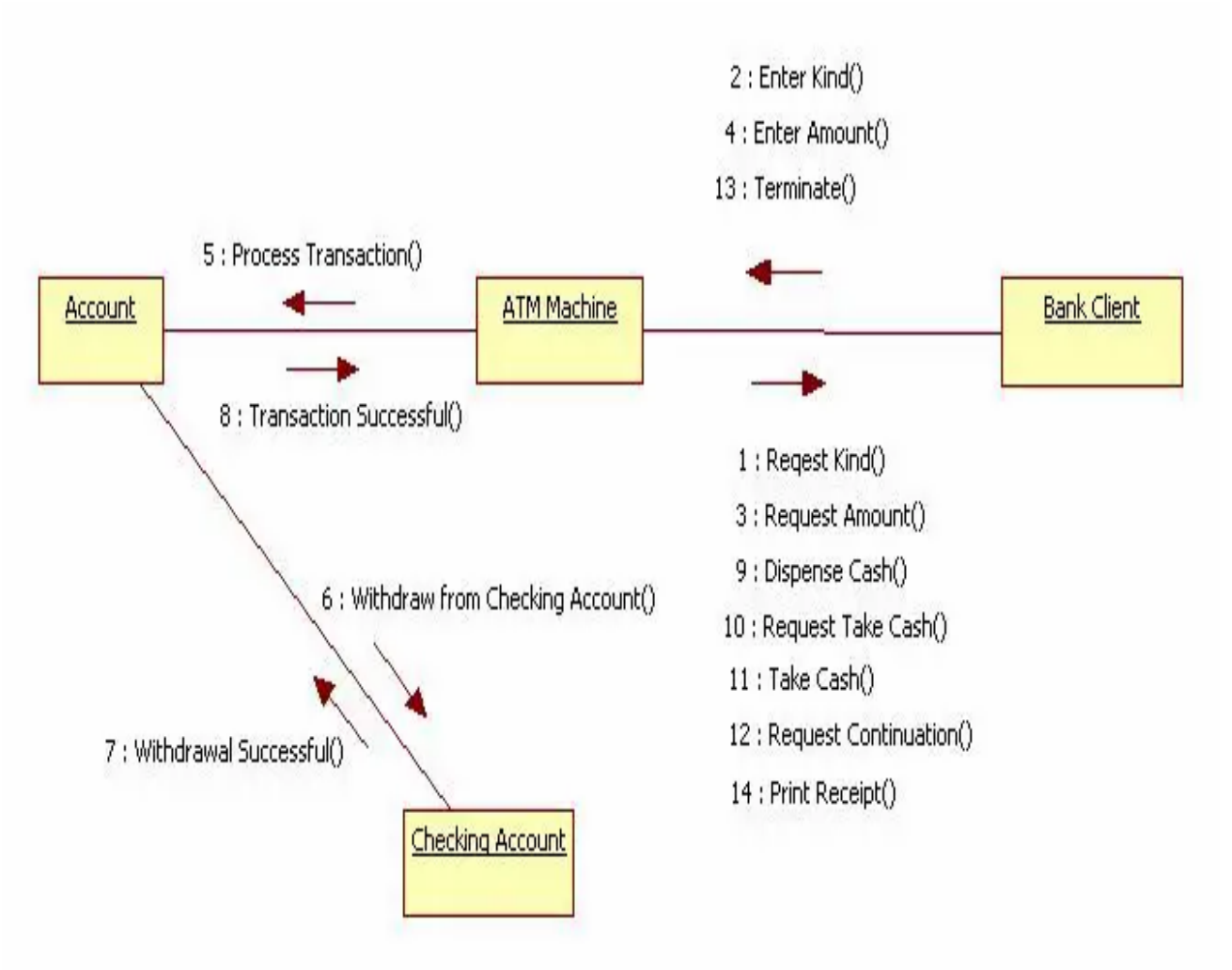
Relationships:

- **ATM uses CashDispenser, CardReader, and Screen.**
- **User interacts with the ATM.**
- **ATM communicates with the Bank.**
- **User has one or multiple Accounts.**
- **Transaction is associated with User and Account.**

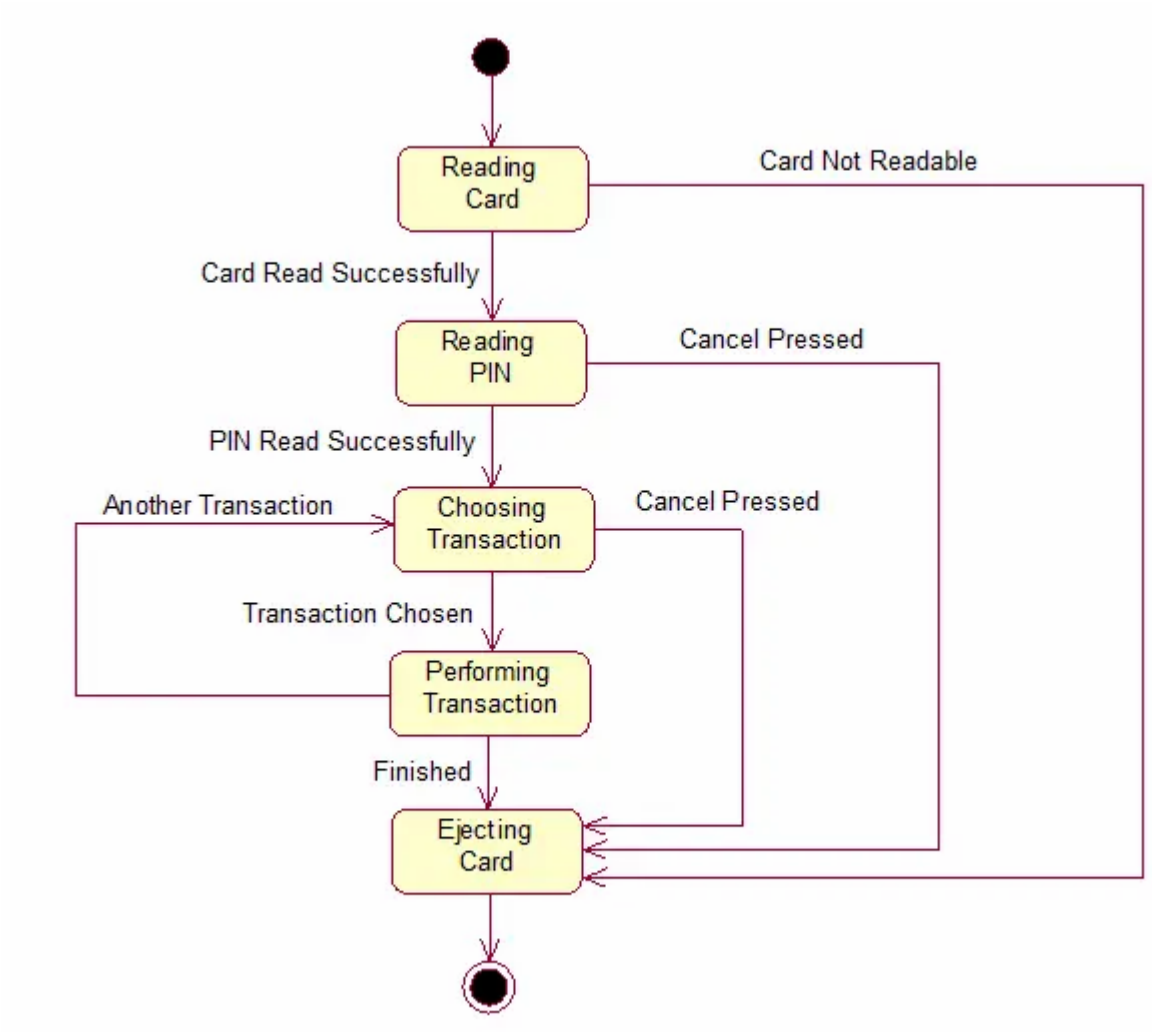


3. Draw the Sequence and Collaboration diagrams for each of the use cases identified.

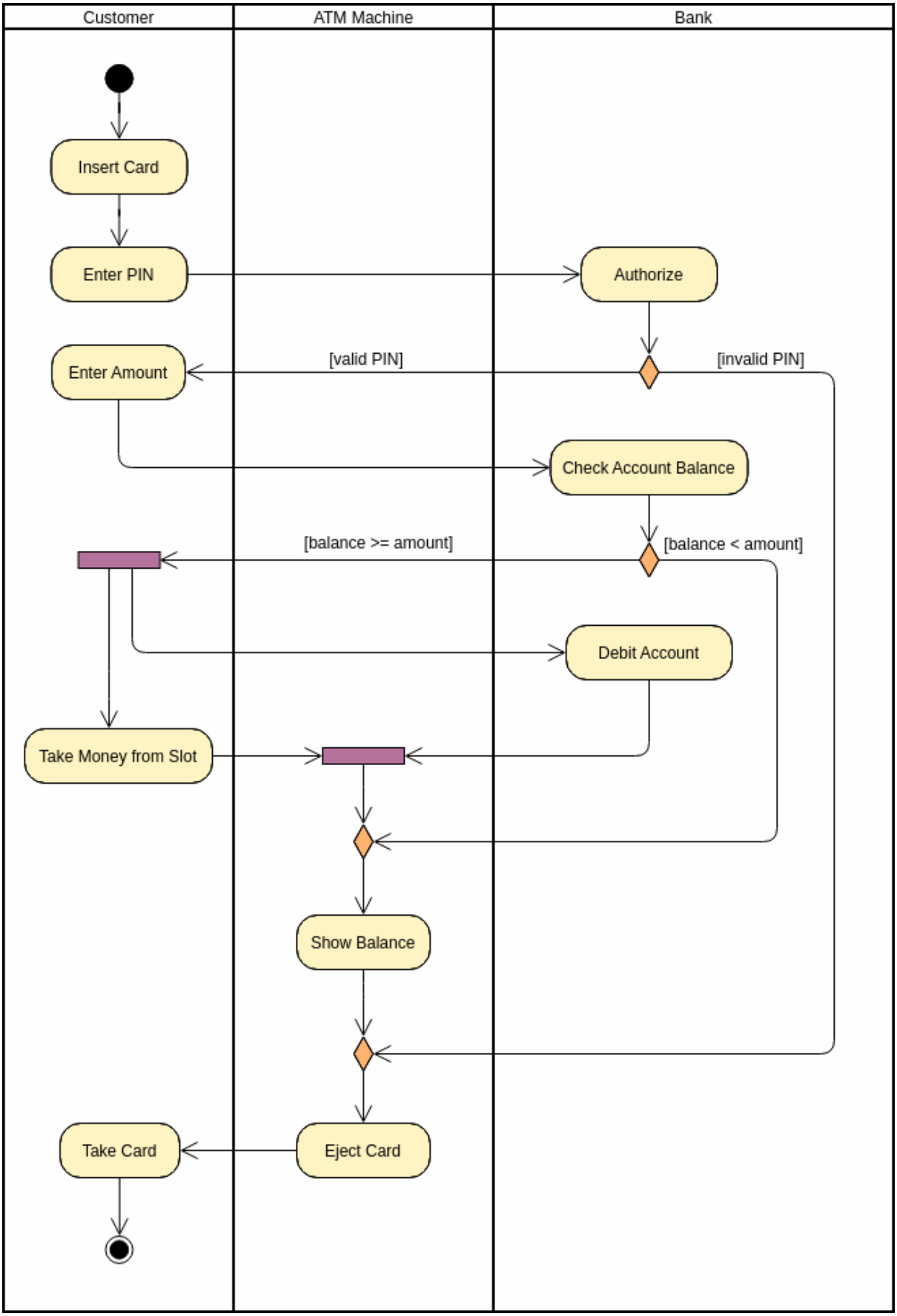

C : Customer
A : ATM
Acc : Account
1: Insert ATM Card
2: Request PIN
3: PIN Entered
4: Verify PIN
5: PIN OK
6: Request Option
7: Option Entered
8: Request Amount
9: Amount Entered
10: processTransaction()
11: Transaction Successful
12: Dispense Cash
13: Request to Take Cash
14: Take Cash
15: Request Continuation
16: Terminate
17: Print Receipt



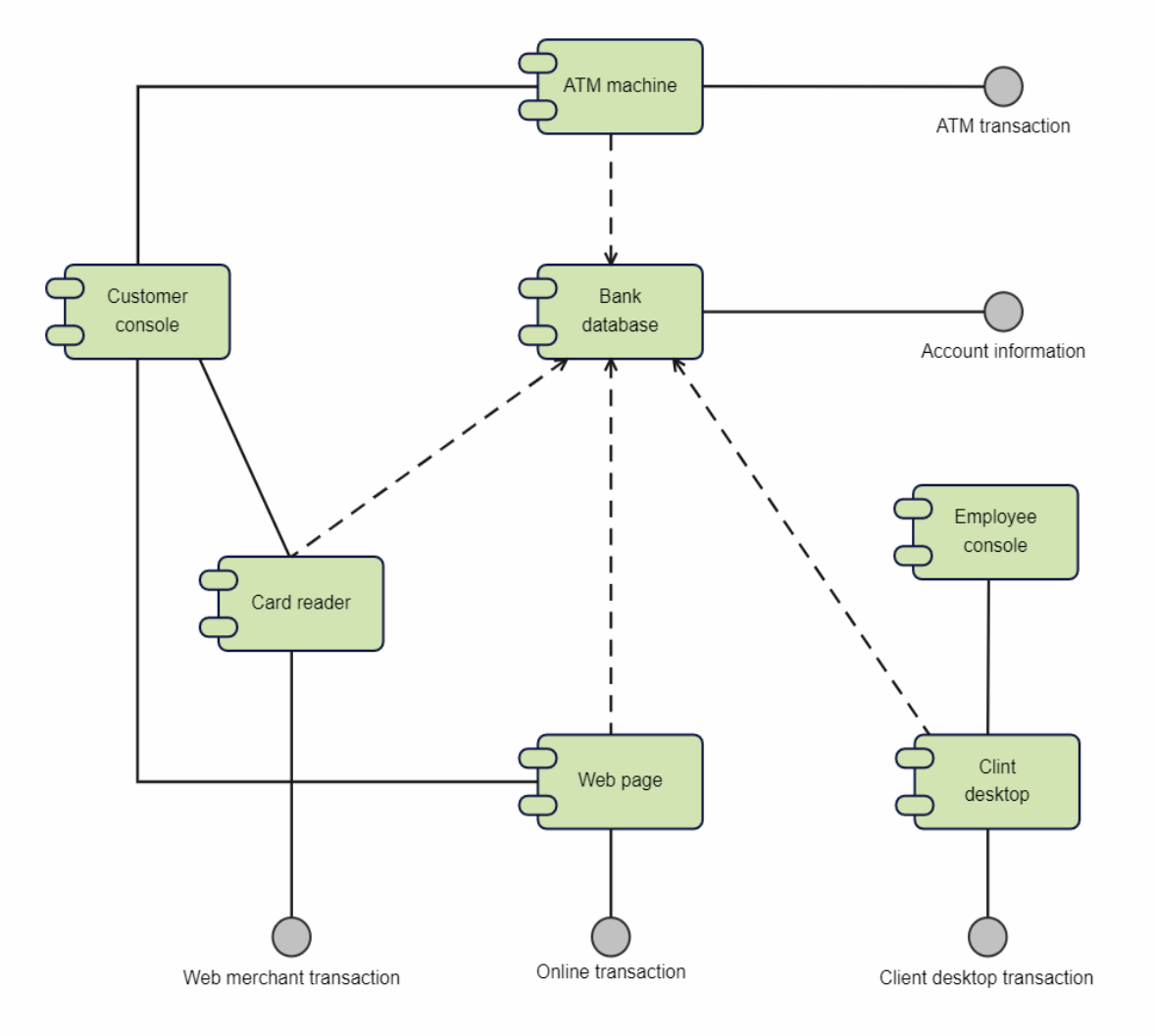
4. Draw the State Chart Diagram for the above system.



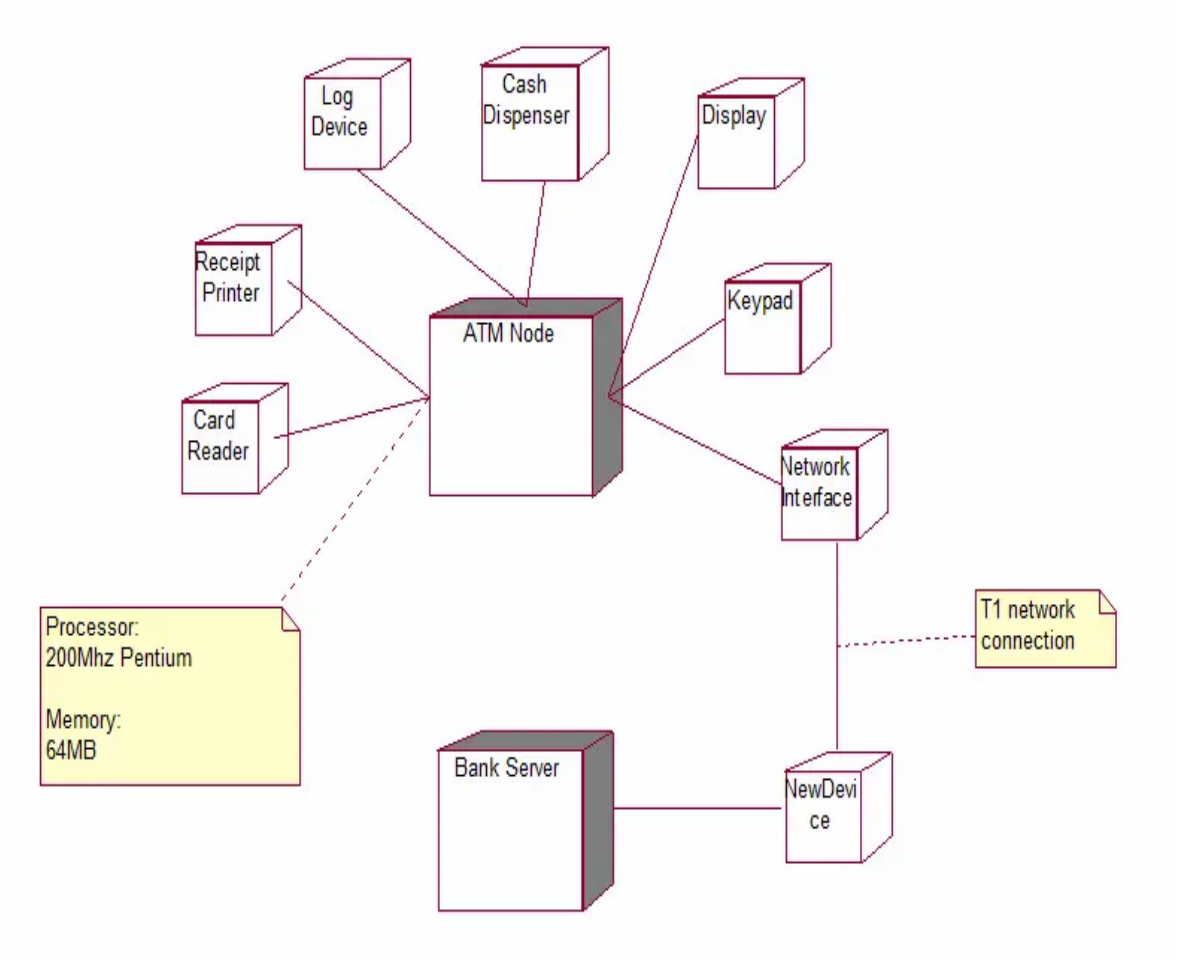
5. Draw the Activity Diagram for the system.


Customer
ATM Machine
Bank
Insert Card
Enter PIN
Authorize
Enter Amount
[valid PIN]
[invalid PIN]
Check Account Balance
[balance >= amount]
[balance < amount]
Debit Account
Take Money from Slot
Show Balance
Take Card
Eject Card

6. Draw the Component Diagram for the system.



7. Draw the Deployment Diagram for the system.


Log Device
Cash Dispenser
Display
Receipt Printer
ATM Node
Keypad
Card Reader
Network Interface
Processor:
200Mhz Pentium
Memory:
64MB
T1 network connection
Bank Server
NewDevi ce